

Project Synopsis

On

“LexAI – AI-Powered Legal Assistant & Contract Intelligence Platform”

Submitted to

Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

[University Logo Placeholder]

for the partial fulfillment of requirement for the Degree of

MASTER OF COMPUTER APPLICATION

Submitted By

Student 1

Student 2

Student 3

Student 4

Guide

[GUIDE NAME]

Department of Computer Science & Information Technology
Chhatrapati Shivaji Maharaj University, Panvel, Navi Mumbai

Academic Year:
2026-27

1. Introduction

The complexity of the legal domain presents significant barriers to entry for both laypeople and professionals. Legal documents, such as commercial contracts, terms of service agreements, non-disclosure agreements, and employment contracts, are notoriously lengthy and densely packed with specialized jargon. This complexity makes it exceptionally difficult for non-lawyers to comprehend their rights, obligations, and the hidden risks buried within the text. Manually reviewing these documents requires immense cognitive effort, hours of reading, and frequently necessitates consulting expensive legal professionals just to grasp the fundamental terms.

LexAI is designed to bridge this accessibility gap. LexAI is an AI-assisted legal information and document intelligence platform that leverages advanced Natural Language Processing (NLP) and Large Language Models (LLMs) to automate the heavy lifting of document review. By allowing users to upload complex legal contracts (in PDF or DOCX format), LexAI automatically extracts, chunks, and processes the text to perform deep legal analysis. The platform instantly provides structured executive summaries, identifies critical clauses, and highlights potential liabilities and obligations.

Beyond static summarization, LexAI empowers users with an interactive, document-based question-answering system. Users can converse directly with their uploaded contracts to locate specific information without reading the entire file. Furthermore, LexAI includes a General Legal Chat feature, providing users with general legal knowledge assistance.

To ensure high accuracy and mitigate the notorious issue of AI "hallucinations," LexAI implements a robust Retrieval-Augmented Generation (RAG) architecture powered by the Google Gemini LLM. By grounding the AI's responses exclusively in the retrieved text of the user's specific document or the curated Legal Knowledge Base, LexAI ensures that the answers are contextually relevant and factual.

Scope of LexAI: The scope encompasses automated executive summarization, localized clause identification, conversational document querying, and curated legal knowledge retrieval. It is important to emphasize that LexAI does not replace human lawyers. The platform is

strictly an AI-assisted information and document analysis system; it provides educational and informational insights and is not a substitute for professional, jurisdictional legal advice.

2. Literature Review

The intersection of artificial intelligence and law has evolved dramatically. Traditional legal document review primarily relied on manual examination by paralegals and junior associates. This process, while highly accurate when performed by experts, is labor-intensive, expensive, and unscalable for the average individual or small business.

Early technological interventions introduced Rule-Based Legal Information Systems and basic Legal Document Management Systems (LDMS). These systems utilized keyword matching (Boolean searches) and rigid heuristic rules to organize and search documents. While they improved retrieval speed, they lacked semantic understanding. A search for "force majeure" would fail if the document instead used the phrase "act of God" or "unforeseeable circumstances."

The advent of Natural Language Processing (NLP) brought about semantic search capabilities. Early NLP-based document analysis tools could extract named entities (parties, dates) and perform basic sentiment analysis, but they struggled with the complex logical reasoning required to interpret contractual obligations.

Recently, Large Language Model (LLM) based legal assistants have transformed the landscape. Models such as GPT-4 and Google Gemini possess deep semantic understanding capabilities, allowing them to summarize dense legalese into plain English. However, applying bare LLMs to the legal domain introduced a critical flaw: hallucination. LLMs generate text based on probabilistic word distributions, meaning they can confidently invent fictitious legal precedents, non-existent clauses, or incorrect statutory interpretations. Furthermore, generic LLMs lack access to private user documents and pose significant privacy concerns if confidential contracts are used as public training data.

To overcome these limitations, Retrieval-Augmented Generation (RAG) systems have become the academic and industry standard. RAG systems combine the reasoning power of LLMs with the factual reliability of Vector

Databases. Research demonstrates that by chunking a document, generating embeddings (e.g., via SentenceTransformers), and storing them in a semantic vector database like ChromaDB, a system can perform highly accurate similarity searches. When a user asks a question, the RAG system retrieves the most mathematically relevant chunks and injects them into the LLM's prompt window. This approach grounds the model's output in verifiable text, drastically reducing hallucinations and enabling it to process documents that exceed the token limits of the LLM.

Despite these advancements, existing RAG implementations often struggle with deterministic failure states, loss of context between chunk boundaries, and poor user-data isolation. LexAI addresses these gaps by implementing strict overlapping text chunking, secure user-specific vector filtering, and a deterministic fallback parser that provides structured answers even when the underlying LLM API is unavailable.

3. Aims & Objective

The primary aim of the LexAI project is to develop a secure, highly accurate, and accessible web-based platform that utilizes Artificial Intelligence and Retrieval-Augmented Generation (RAG) to demystify complex legal documents and provide verified legal information.

Implemented Objectives:

- User Authentication: Develop a secure registration, login, and session management system using Flask-Login and Bcrypt.
- Document Processing: Enable secure upload and raw text extraction from PDF and DOCX formats utilizing PyMuPDF and python-docx.
- Document Statistics: Automatically calculate and store metrics such as word count, character count, and page count.
- AI Legal Analysis: Prompt the Gemini API to automatically generate an Executive Summary, highlighting key clauses, obligations, and risks.
- RAG-Based Retrieval: Implement a complete vector embedding pipeline using SentenceTransformers (all-MiniLM-L6-v2) and ChromaDB to semantically index documents.
- Document Chatbot: Create a conversational interface allowing users to query their specific uploaded contracts, backed by RAG.

- General Legal Chat & Knowledge Base: Provide an admin-curated legal Knowledge Base to ground general legal queries safely.
- Secure User Isolation: Enforce strict database and vector-store filtering to ensure users can only access and query their own private documents.
- Deterministic Fallback: Implement a robust parser that gracefully handles Gemini API failures (503/429) by retrieving and formatting raw Knowledge Base content without relying on the LLM.

Future Objectives (Planned):

- Optical Character Recognition (OCR) for processing scanned, image-based legal documents.
- Multilingual support to analyze and translate legal concepts into regional languages (e.g., Hindi, Marathi).
- Migration to persistent cloud infrastructure for scalable deployment.

4. Problem Definition

The legal ecosystem is largely inaccessible to the general public due to the sheer volume and complexity of legal text. When an individual or small business is presented with a standard contract, they face several immediate challenges:

1. Manual Review Effort: Contracts are often tens or hundreds of pages long. Reading and comprehending these documents requires a prohibitive amount of time.
2. Obfuscated Clauses and Hidden Risks: Critical obligations, termination conditions, and liability waivers are frequently buried within dense, archaic legal terminology (legalese). Identifying these risks without professional training is highly error-prone.
3. Information Retrieval Friction: Even when a user knows what they are looking for (e.g., "What is the penalty for late payment?"), finding the exact clause in a massive document is tedious.
4. Unreliable AI Solutions: While public generative AI chatbots are available, they provide generic, ungrounded responses. If a user asks a public AI about their contract, the AI may hallucinate standard legal practices instead of citing the actual terms of the user's specific document. Furthermore, pasting confidential contracts into public AI interfaces poses severe privacy and security risks.

5. Lack of Centralized Intelligence: There is a lack of platforms that securely combine private document analysis, conversational querying, and curated general legal knowledge in one isolated, user-friendly environment.

LexAI addresses this problem by providing a secure, isolated platform where documents are processed locally into a vector database, and an LLM is strictly constrained (via RAG) to answer questions based only on the user's uploaded text or the platform's curated knowledge base.

5. Proposed Model

The proposed LexAI system follows a modern, decoupled client-server architecture utilizing a Flask backend, a MySQL relational database for metadata, and a local ChromaDB instance for semantic vector storage. The system integrates external intelligence via the Google Gemini API.

System Architecture Flow:

User → Web Interface (HTML/JS/Bootstrap) → Flask Backend → (Authentication / Document Management / AI Analysis / Chat) → MySQL + Local File System + ChromaDB → Gemini AI

- Frontend Layer: Built with HTML5, CSS3, and Vanilla JavaScript. It utilizes Bootstrap 5 for responsiveness, marked.js for converting AI Markdown to HTML, DOMPurify to prevent XSS attacks, and MathJax to render complex logic or equations output by the AI.
- Flask Backend: The core Python application that handles HTTP routing, session management (Flask-Login), and business logic. It securely isolates user data and orchestrates the document processing pipeline.
- MySQL Database: Stores relational data including user credentials (hashed via Bcrypt), document metadata (file paths, word counts), pre-generated AI executive summaries, and complete chat conversation histories.
- Document Processing & Storage: Uploaded PDFs and DOCX files are stored on the local filesystem. PyMuPDF and python-docx extract the raw unicode text from these files.
- Vector Embeddings & ChromaDB: The extracted text is passed to a local SentenceTransformers model (all-MiniLM-L6-v2) which converts

text chunks into 384-dimensional mathematical vectors. These vectors are stored in ChromaDB, enabling semantic similarity search.

- **Retrieval-Augmented Generation (RAG) & Gemini API:** When a user queries a document, the backend searches ChromaDB for the most semantically relevant text chunks. These chunks are appended to a strict system prompt and sent to the Google Gemini API, which generates a natural language response grounded entirely in the provided chunks.

6. Methodology

The development and operational methodology of LexAI is structured into a logical pipeline that handles everything from secure access to complex AI generation.

- **A. User Registration & Login:** Users create accounts. Passwords are salted and hashed using Bcrypt. Flask-Login manages secure HTTP-only sessions.
- **B. Document Upload:** Users upload legal documents (.pdf or .docx) via a secure multipart form. The backend validates file types and size limits.
- **C. Text Extraction & Statistics:** PyMuPDF (for PDFs) and python-docx (for Word docs) strip formatting and extract raw text. The system calculates and stores word and page counts.
- **D. Text Chunking:** Because LLMs have token limits, the document is sliced into smaller chunks (e.g., 500 words) with overlapping windows (e.g., 50 words) to ensure context is preserved across chunk boundaries.
- **E. Embedding & ChromaDB Storage:** Each chunk is converted into a vector using the local all-MiniLM-L6-v2 model and stored in ChromaDB, strictly tagged with the user's ID and document ID to enforce data isolation.
- **F. Gemini-Based Legal Analysis:** Immediately after processing, the text is sent to the Gemini API with a strict JSON-schema prompt to generate an Executive Summary and highlight key clauses. This analysis is cached in MySQL.

RAG Retrieval & Chat Workflow:

1. **User Question:** The user types a question in the chat interface.

2. Semantic Search: The question is embedded into a vector. ChromaDB calculates the mathematical distance between the question and all stored document chunks.
3. Relevant Chunks: ChromaDB returns the top matching chunks.
4. Context Construction: The chunks are combined into a prompt: "Answer the question based ONLY on the following text..."
5. Gemini Generation: Gemini returns a factual, grounded response.

Distinction Between Chat Modes:

- Document Chat: The semantic search is filtered strictly by the selected document ID. It only retrieves content belonging to that specific uploaded contract.
- General Legal Chat: The semantic search ignores user documents and filters exclusively by `source_type: knowledge_base`, retrieving answers from the admin-curated legal repository.

Deterministic Fallback Mechanism:

If the Gemini API becomes unavailable (e.g., HTTP 503 or 429 Quota Exceeded), the RAG pipeline intercepts the failure. Because ChromaDB has already successfully retrieved the relevant Knowledge Base chunks, a deterministic Python fallback parser extracts the raw legal information, strips database separators, formats it cleanly in Markdown, and returns it to the user. This ensures the system remains functional even without the LLM.

7. Plan of Work

The project was structured into distinct development phases:

Phase 1: Project Setup & Authentication (Implemented)

- Activities: Set up Flask environment, configure MySQL database, implement user models, and build registration/login systems.
- Technologies: Python, Flask, Flask-SQLAlchemy, Bcrypt, HTML/CSS/JS.

Phase 2: Document Upload & Text Extraction (Implemented)

- Activities: Implement secure file uploads, validate extensions, and integrate parsing libraries to extract raw text and calculate document

statistics.

- Technologies: PyMuPDF (fitz), python-docx, Werkzeug.

Phase 3: AI Legal Document Analysis (Implemented)

- Activities: Integrate Google Gemini API, engineer structured JSON prompts, and create the frontend Analysis Dashboard.
- Technologies: google-genai SDK, Bootstrap 5.

Phase 4: RAG & Document Chat (Implemented)

- Activities: Deploy local embedding models, configure ChromaDB, implement text chunking logic, and build the asynchronous chat interface.
- Technologies: SentenceTransformers, ChromaDB, AJAX, marked.js, DOMPurify.

Phase 5: Knowledge Base & General Legal Chat (Implemented)

- Activities: Build the Admin Dashboard, implement KB CRUD operations, create the seed_kb.py generation script, and isolate chat routes.
- Technologies: Flask Blueprints, Gemini API.

Phase 6: Testing, Security & Optimization (Implemented)

- Activities: Write comprehensive Pytest suites (86 passing tests), secure routes against path traversal, implement MathJax for frontend rendering, and build the deterministic 503 fallback parser.
- Technologies: pytest, MathJax.

Phase 7: Cloud Deployment & OCR (Future Scope)

- Activities: Migrate local ChromaDB and file storage to cloud infrastructure, and implement OCR for image-based PDFs.
- Technologies: AWS/GCP, Tesseract OCR.

8. Conclusion

The LexAI project successfully demonstrates the immense potential of combining Large Language Models with Retrieval-Augmented Generation to solve the accessibility crisis in legal document comprehension. By integrating an intuitive web interface with a robust, locally embedded RAG pipeline, LexAI provides a highly accurate, secure, and isolated environment for contract intelligence.

Through automated executive summaries, clause breakdowns, and a dual-mode conversational chatbot, the platform significantly reduces the time

and cognitive load required to understand dense legal text. The strict separation of the Document Chat and the curated General Legal Knowledge Base ensures that the AI's responses are reliably grounded, directly mitigating the risks of model hallucination.

Furthermore, the implementation of comprehensive security measures, user isolation, and a deterministic fallback parser guarantees that LexAI is a resilient, production-ready prototype. While LexAI explicitly operates as an AI assistant rather than a substitute for professional legal counsel, it represents a significant technological leap toward making legal information transparent, accessible, and understandable for everyone.

9. References

- [1] Flask Documentation. "Flask: A Python Microframework." Available at: <https://flask.palletsprojects.com/>
- [2] Google GenAI Documentation. "Gemini API Overview." Available at: <https://ai.google.dev/docs>
- [3] ChromaDB. "Chroma: The AI-native open-source vector database." Available at: <https://docs.trychroma.com/>
- [4] Reimers, N., & Gurevych, I. (2019). "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks." Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing.
- [5] Lewis, P., et al. (2020). "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." Advances in Neural Information Processing Systems (NeurIPS).
- [6] PyMuPDF Documentation. "fitz - PyMuPDF 1.23.0 documentation." Available at: <https://pymupdf.readthedocs.io/>
- [7] Python-docx Documentation. "python-docx: Create and update Microsoft Word .docx files." Available at: <https://python-docx.readthedocs.io/>
- [8] SQLAlchemy Documentation. "SQLAlchemy: The Database Toolkit for Python." Available at: <https://www.sqlalchemy.org/>

Prof. [GUIDE NAME]

Guide.

CS/IT Department

Dr. Praveen Gupta

H.O.D.

CS/IT Department